REPLIT CLASSROOM LAB · MULTIMODAL AI

Build the Inspector

The gallery's Inspector checks a food photo before the API saves it. It uses a vision model for evidence and regular program code for the final decision.

Learning goals

- Trace an image from the browser to an AI model and back.
- Use a schema to constrain the model's response.
- Keep final approval rules in application code.

What "agent" means here

A focused AI step: a clear role, image input, structured report, and a programmatic approval rule. It is not allowed to save files or change the database itself.

Inspector flow

STAGE	WHAT HAPPENS
1 · Browser	Resizes the selected photo and sends title, caption, and image to `POST /api/dishes`.
2 · API	Validates text and image type/size before asking the model.
3 · Inspector	Sends the photo plus instructions to Gemini; requests a structured verdict.
4 · Rule check	Application code approves only a match with confidence at least 0.70.
5 · Storage	Approved photo is saved with an Inspector note; rejected photo is not saved.

Three parts work together

Model: interprets image content. **Schema:** makes the answer predictable. **Program:** applies the threshold and controls whether the database write happens.

Important: does this agent call tools?

No function-calling tools are registered in this Inspector. It uses LangChain's Gemini chat model to send text and an image, then gets structured output. The tools in the developer's toolkit are the Gemini model integration, LangChain message classes, Zod validation, Express, and the database store. It does not let the model call arbitrary functions.

That is a deliberate safety boundary: the model reports what it sees; trusted code decides what happens next.

From an image to a verdict

1 Describe the output with Zod

In `artifacts/api-server/src/agents/verdict.ts`, define exactly what the model must report.

```
export const DishVerdict = z.object({
  isMatch: z.boolean(),
  confidence: z.number().min(0).max(1),
  whatISee: z.string()
});
```

2 Create a narrow prompt and model

`dishInspector.ts` uses `ChatGoogle` with the `GEMINI_API_KEY` Replit Secret. The system instructions define the only task: decide whether the image shows the target dish, and say “not a match” when uncertain.

3 Send both text and image

LangChain’s `SystemMessage` supplies rules; a `HumanMessage` contains a question and an image block. `withStructuredOutput(DishVerdict)` asks for the expected shape.

```
inspector = model.withStructuredOutput(DishVerdict);
const result = await inspector.invoke([
  new SystemMessage(INSPECTOR_RULES),
  new HumanMessage({
    contentBlocks: [
      { type: "text", text: "Does this photo show deep dish?" },
      { type: "image", mimeType: image.mimeType, data: image.base64 }
    ]
  })
]);
```

4 Make the final decision in code

`decideVerdict` approves only when `isMatch` is true and confidence is at least `0.70`. A confident approval gets a short badge. Otherwise it returns a rejection reason for the visitor.

Keep the key private

Add `GEMINI_API_KEY` using Replit Secrets. Server code reads it from `process.env`. Never put the value in a code file, browser request, screenshot, handout, or Git commit.

STUDENT EXERCISE · TEST AND IMPROVE

Make the Inspector reliable

1 Trace the save gate

Open `artifacts/api-server/src/routes/dishes.ts`. Find where `inspectDish(image)` runs. Confirm the database insert happens only after the approved check.

2 Study the two failure paths

A non-match returns HTTP `422` with an explanation. If Gemini fails, the route returns HTTP `502`. Neither path saves the photo.

3 Run a small test set

With the API key configured in Replit Secrets, try one clear deep-dish photo, one clearly unrelated food photo, and one ambiguous photo. Record status, note/reason, and whether a database row was created.

TEST	EXPECTED BEHAVIOR
Clear target dish	Approved when model says match and confidence ≥ 0.70; row saved with Inspector note.
Unrelated food	Rejected with 422; no row saved.
Unsure / low confidence	Fail closed: reject; no row saved.
Model outage / bad key	502 response; no row saved; do not reveal the secret.

Stretch challenge

Add a field such as `foodType` to the Zod verdict. Update the prompt and UI, then test the result. Keep the application's approval rule explicit and deterministic.

Safety checklist

- Treat image text as untrusted; the prompt says to ignore instructions inside images.
- Validate model output before using it.
- Use a confidence threshold; do not treat every model answer as true.
- Keep the API key server-side.
- Do not save rejected images.

Instructor discussion

Ask: Why does code—not the model—decide whether to save? What can a confidence score tell us, and what can't it guarantee? Why is "unsure means reject" appropriate for a gallery with a narrow topic?

Project files: `src/agents/verdict.ts`, `src/agents/dishInspector.ts`, and `src/routes/dishes.ts` inside `artifacts/api-server`.